

Day 2

Multi dimensional arrays

→ Array traversal is contiguous as array memory is stored linearly.

```
void print (int *arr, int rows, int cols)
{
    for (i=0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            printf ("%d", *(arr + i * cols + j))
        }
        printf ("/n")
    }
}

int arr[3][3] = { . . . . };
print ((int *)arr), 3, 3]
```

→ This is possible but not preferred.

→ Two types of main orders, Row Major Order, Column major Order.

	0	1	2		Row Order	Column Order
	← j →					
↑	0	A	B	C	A	A
	1	D	E	F	B	D
i	2	G	H	I	C	G
↓	3	J	K	L	D	J
					E	B
					F	E
					G	H
					H	K
					I	C
					J	F
					K	I
					L	L

malloc

int arr;



already a pointer but requires initialization for the columns.

So what if we make a pointer to a pointer?

int** arr;

arr = (int**) malloc (row * sizeof(int*))



or whatever type user enters.

I assume int here.

```
for (i=0; i < row; i++)  
{
```

```
    arr[i] = (int *) malloc (col * size of (int));
```

```
}
```

This should in theory, create a memory map for a 2D array.

→ WORKS !!!

Same can be done with calloc but instead of multiply, it takes as argument count of contiguous allocation.

```
arr = (int **) calloc (r, size of (int *));
```

```
arr[i] = (int *) calloc (c, size of (int));
```

Function Stacks

→ Every Function creates its own stack frame and pushes it into the main stack.

→ Local variables remain, until stack frame is not popped

↳ Note: This does not apply if you use malloc and not run free. If memory is orphaned memory leak will occur.

Typedef

- ↳ Creates an alias for an existing datatype
- ↳ Basically say `typedef int Integer`

so "Integer" and "int" mean the same thing.

Struct

- ↳ Seems similar to classes, but isn't the case.
- ↳ Structs can't have functions.